CONF42 PLATFORM ENGINEERING 2026

Building the Internal Developer Platform for Generative AI

From raw foundation models to governed, self-service APIs

Brindal Patel

Senior Software Engineer

Views are my own. Not affiliated with or endorsed by any company.

A FAMILIAR STORY

A retry loop, a shared API key, and one very expensive night.

Nobody could tell which service did it, or stop it without breaking every other team.

THE IDEA IN ONE LINE

An IDP is a paved road: the easy way is also the safe way.

model-access.yaml (one pull request)

```
apiVersion: genai.platform/v1
kind: ModelAccess
metadata:
  name: support-bot
spec:
  team: support
  models: [tier1-chat, internal-small]
  budget: team-default
  dataClass: internal
```

A model alias

not a vendor SDK and a key pasted into the repo

A budget with an owner

spend is visible, capped and charged back per team

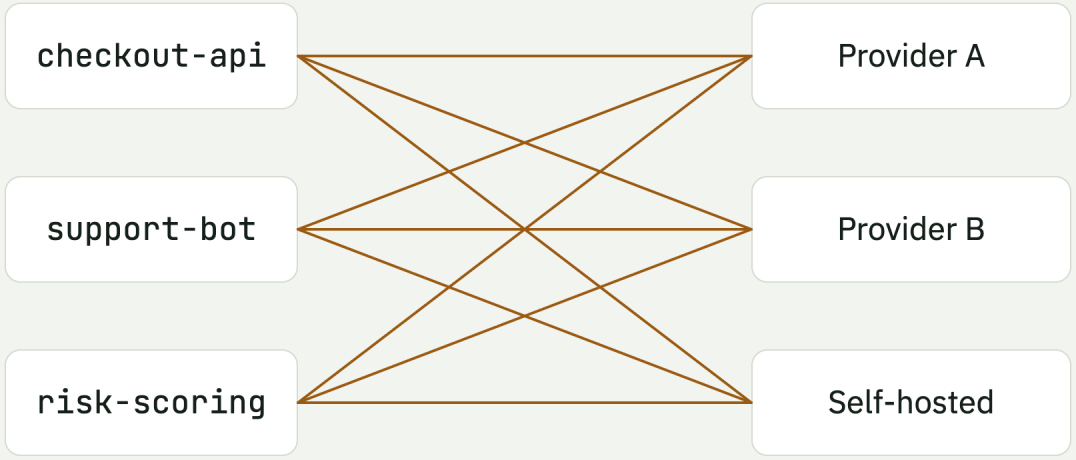
A trace for every call

who called, which model, how many tokens, which policy said yes

WHY NOW

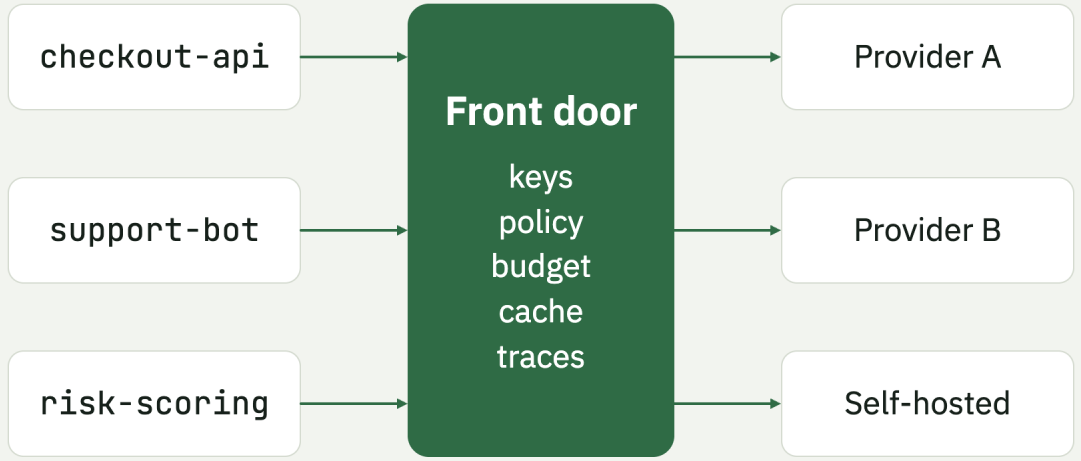
From N × M integrations to one front door

Today: every team wires its own client



Keys in repos. No spend owner. Uncoordinated 429s. The same prompt paid for again and again.

With a platform: one front door



One key per team, one budget, one trace, and one place to change models.

THE REQUEST PATH

Six stages every model call travels

01
Identity
mTLS, and the tenant comes from the token

02
Redact
PII and secrets out before anything leaves
`Envoy ext_proc`

03
Decide
OPA says allow, degrade or deny, and places a budget hold
`OPA`

04
Cache
scoped per tenant, and checked for entities

05
Route
to a model alias, with eval-gated failover

06
Settle
capture the real tokens, release the rest

Cache hit at stage 04: return right away and release the whole hold. No upstream call, no tokens charged.

Every stage emits an OpenTelemetry span, tied to the tenant and its Kubernetes namespace.

Agent tool calls (MCP) use the same door: the same policy checks the tool name and its arguments.

IDEA 1 · BUDGETS

Tokens are money, so budget them like a card payment

Step 1

Authorize

Card: the bank holds the amount before the sale.

LLM call: hold the input cost plus the cost of max output tokens.

Step 2

Capture

Card: the merchant settles the real total.

LLM call: charge the tokens the response actually reports.

Step 3

Release

Card: any unused hold drops off.

LLM call: return the remainder to the team's budget. A cache hit releases all of it.

Why it works

Output tokens are unknown until the call ends, and ten parallel requests can all pass a "spend is under budget" check at once. A hold counts the moment it is placed, so the budget cannot be raced or overrun.

IDEA 2 · CACHE CORRECTNESS

Similar is not the same when money is involved

“Send $100 to my savings”

“Send $1,000 to my savings”

Nearly identical vectors. Very different answers.

“Balance of account [ID]?”

After redaction, this is the same text for every user. A naive cache serves user A’s answer to user B.

1 **Same scope?**
tenant, model alias, system prompt and decoding settings

2 **Close enough?**
similarity over a threshold tuned for your embedding model

3 **Same entities?**
entity hashes match: amounts, accounts, tickers, dates, negations

4 **Cacheable intent?**
transactional and per-user prompts never hit the cache

All four yes: serve the hit and sample it for shadow checks. Any no: call the model.

IDEA 2, CONTINUED

The caching ladder

Climb only as high as your correctness checks can follow.

Rung 1

Provider prompt cache

Reuses long shared prefixes. Almost no correctness risk.

Rung 2

Exact-match cache

Same normalized request, same answer. Easy to reason about.

Rung 3

Scoped semantic cache

Catches paraphrases, but only behind the four-check gate.

Rung 4

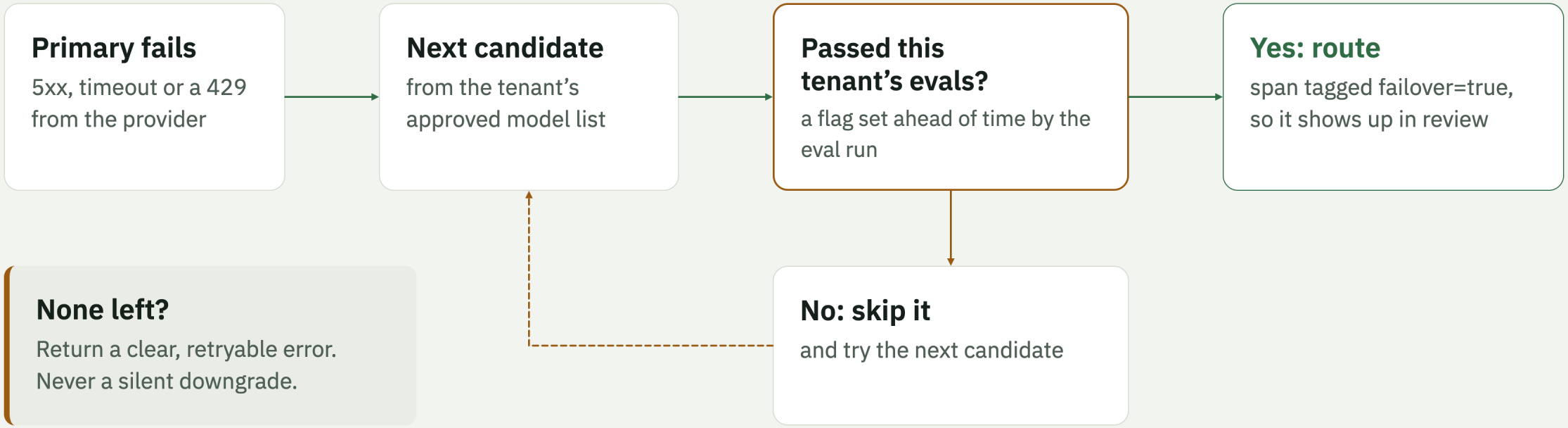
KV-cache-aware routing

Self-hosted only: send each request where its prefix is already warm.

IDEA 3 · FAILOVER

Fail over only to models you have tested

Evals run in CI on every model change. The router only checks a “passed” flag.



POLICY AS CODE

One decision: allow, degrade or deny

```
package genai.gateway

default decision := {"action": "deny"}

allowed if input.alias in data.tenants[input.tenant].models
clean if count(input.dlp.findings) == 0
fits if input.hold <= data.budgets[input.tenant].available
may_degrade if data.tenants[input.tenant].allow_degrade

decision := {"action": "allow", "route": input.alias} if {
  allowed; clean; fits
}
decision := {"action": "degrade", "route": "internal"} if {
  allowed; clean; not fits; may_degrade
}
```

One decision object

Rules cannot contradict each other, and the gateway reads one answer.

Redaction runs first

A DLP filter in Envoy ext_proc scrubs the body. OPA only reads its findings.

Budgets use the hold

The check is the worst-case cost of this request, not spend so far.

`Written for OPA 1.x`

TELEMETRY AND CHARGEBACK

One trace tells you who spent what, and why

```
POST /v1/chat SERVER, tenant=payments-api
├─ dlp.scan findings=0
├─ policy.evaluate decision=allow
├─ budget.hold placed
├─ cache.lookup result=miss
├─ chat tier1-chat CLIENT
│   gen_ai.operation.name chat
│   gen_ai.provider.name openai
│   gen_ai.request.model tier1-chat
│   gen_ai.usage.input_tokens
│   gen_ai.usage.output_tokens
└─ budget.capture remainder released
```

Name it the OTel way

Model spans are “{operation} {model}” with gen_ai.* attributes.

Put the owner on the span

The calling tenant goes on each request span; the gateway’s namespace stays on the resource. That is your chargeback.

Pin the version

The GenAI conventions are still in Development status and do change.

WHERE TO START · MY TAKE

Impact versus effort

↑ Higher impact

Quick wins: start here

One gateway with per-team keys
Provider prompt caching
Token attribution in traces

Big bets: plan for these

Hold, capture and release budgets
Scoped semantic cache with the gate
Eval-gated failover

Fill-ins: when there is time

Per-team cost dashboards
Budget alerts in team chat

Avoid early

Writing a gateway from scratch
Training your own embedding model

↓ Lower impact

← Lower effort

Higher effort →

ANTI-PATTERNS

Four mistakes worth catching in review

Don’t

One cache shared by every tenant

Do: scope keys by tenant, model alias, system prompt and settings.

Don’t

Check budgets after the call

Do: place a hold before the call and capture after it.

Don’t

Fail over to any model that answers

Do: fail over only to models that pass the tenant’s evals.

Don’t

Embed prompts before redaction

Do: redact first, and treat redacted prompts as per-user or uncacheable.

TAKE HOME

Treat models like infrastructure, and tokens like money

1

Govern tokens like money

Hold, capture, release. Budgets that cannot be raced.

2

Cache only what is provably the same

Scope, similarity, entities and intent. All four, every time.

3

Fail over only to tested models

A backup earns its place by passing your evals.

Articles and talk notes

`brin-builds.pages.dev`

Thank you

Brindal Patel